conversion. For negative numbers, the following algorithm is often used: (1) take the absolute value of the decimal number, (2) convert it to binary, (3) complement all of the bits (ones become zeros and zeros become ones), (4) add 1 to the binary number. For example: -5 → 5 → 0101 → 1010 → 1011. Two's complement is hard for humans, but easy for digital electronics.

Floating Point (Real Numbers)

The encoding scheme for floating point numbers is more complicated than for fixed point. The basic idea is the same as used in scientific notation, where a **mantissa** is multiplied by ten raised to some **exponent**. For instance, 5.4321×10^6, where 5.4321 is the *mantissa* and 6 is the *exponent*. Scientific notation is exceptional at representing very large and very small numbers. For example: 1.2×10^{50}, the number of atoms in the earth, or 2.6×10^{-23}, the distance a turtle crawls in one second, compared to the diameter of our galaxy. Notice that numbers represented in scientific notation are *normalized* so that there is only a single nonzero digit left of the decimal point. This is achieved by adjusting the exponent as needed.

Floating point representation is similar to scientific notation, except everything is carried out in base two, rather than base ten. While several similar formats are in use, the most common is ANSI/IEEE Std. 754-1985. This standard defines the format for 32 bit numbers called **single precision**, as well as 64 bit numbers called **double precision**. As shown in Fig. 4-2, the 32 bits used in single precision are divided into three separate groups: bits 0 through 22 form the mantissa, bits 23 through 30 form the exponent, and bit 31 is the sign bit. These bits form the floating point number, *v*, by the following relation:

EQUATION 4-1
Equation for converting a bit pattern into a floating point number. The number is represented by *v*, *S* is the value of the sign bit, *M* is the value of the mantissa, and *E* is the value of the exponent.

$$v = (-1)^S \times M \times 2^{E-127}$$

The term: $(-1)^S$, simply means that the sign bit, *S*, is 0 for a positive number and 1 for a negative number. The variable, *E*, is the number between 0 and 255 represented by the eight exponent bits. Subtracting 127 from this number allows the exponent term to run from 2^{-127} to 2^{128}. In other words, the exponent is stored in *offset binary* with an offset of 127.

The mantissa, *M*, is formed from the 23 bits as a *binary fraction*. For example, the decimal fraction: 2.783, is interpreted: 2 + 7/10 + 8/100 + 3/1000. The binary fraction: 1.0101, means: 1 + 0/2 + 1/4 + 0/8 + 1/16. Floating point numbers are *normalized* in the same way as scientific notation, that is, there is only one nonzero digit left of the decimal point (called a *binary point* in